

The Redemption of the Driver

How Experience Determinants Became Computable

author: "Raheem Asghar"

date: "December 2025"

Abstract

For over two decades, customer experience (CX) practitioners have organized their work around "drivers": the factors that determine satisfaction, dissatisfaction, loyalty, and disloyalty. Yet despite universal acceptance of this concept, experience drivers have never existed as computational entities. They floated as thematic labels, drifted across analysts and time periods, and remained external to the systems meant to act on them.

This document presents the architectural breakthrough that changes this: the instantiation of experience drivers as persistent, queryable system state. Through three core decisions preserving full behavioral context, building comprehensive state structures, and generating atomic missions from granular evidence experience drivers gain the computational substrate they have always lacked.

The result is not better analytics. It is infrastructure: The Experience State Layer, enforced by the Emotional Relational Model (ERM), enabling experience determinants to participate in computation for the first time.

I. The 25-Year Problem

What CX Has Always Known

Customer experience professionals have always understood that certain factors drive outcomes:

- "What are the drivers of satisfaction?"
- "Which drivers impact NPS most?"
- "We need to improve our top 3 drivers"

The language was correct. The intuition was sound.

Drivers exist in reality. When customers churn, there are reasons. When satisfaction improves, there are causes. These determinants are not abstractions they are operational surfaces where experience is shaped:

- Refund cycle time
- Appointment scheduling reliability
- Call center wait times
- Product availability
- Billing clarity
- Triage efficiency

The CX field has spent 20+ years discussing, measuring, and analyzing these drivers. But drivers have never been real inside systems.

What Was Broken

Experience drivers existed only as:

1. Thematic Labels (created per analysis)

- Analyst interpretation, not canonical identity
- Changed across quarters: "Delivery Speed" → "Shipping Time" → "Fulfillment Timeliness"
- No continuity, no single source of truth

2. Dashboard Categories (reset every period)

- Each analysis started from scratch
- No memory of previous state
- No lineage of what changed or why

3. Insight Outputs (commentary, not computation)

- "Top drivers this quarter: Wait Times, Communication, Billing"
- Observed, not operated on
- Summarized, not instantiated

4. Generic Recommendations (vague and unmeasurable)

- "Improve communication"
- "Reduce wait times"
- "Enhance transparency"

Result:

- Drivers drifted across time and analysts
- The same issues were re-analyzed endlessly
- No one knew what interventions actually worked
- Actions remained vague and unmeasurable
- CX became perpetual commentary, never computation

The Traditional Flow (Everyone Else)

...

Raw Feedback

↓

Extract themes/topics (context lost)

↓

Count frequency + sentiment (positive/negative binary)

↓

Generate insight report ("Top 5 Drivers This Quarter")

↓

Recommend generic actions ("Improve delivery speed")

↓

[No memory, no state, no lineage, no follow-through]

↓

Repeat next quarter from scratch

...

What Gets Lost at Every Step

At Theme Extraction:

- ❌ Behavioral context (WHY was delivery speed an issue?)
- ❌ Situational nuance (Was it late delivery? Missing delivery? Damaged goods?)
- ❌ Interaction moment (When in the journey did this occur?)

At Frequency + Sentiment:

- ❌ Emotional intensity (Anger vs. Agitation vs. Fear—all collapse to "negative")
- ❌ Volatility signal (Are emotions mixed or unified?)
- ❌ Temporal patterns (Is this getting worse? Stable? Improving?)

At Recommendation:

- ❌ Execution clarity (What EXACTLY should we do?)
- ❌ Behavioral specificity (Which sub-problem under "delivery" matters most?)
- ❌ Evidence linkage (Which customer voices support this?)

At Follow-Through:

- ❌ Outcome tracking (Did the action work?)
- ❌ State comparison (What was the before/after?)
- ❌ Institutional memory (What worked historically?)

The Core Deficiency:

Experience drivers had:

- ❌ No permanent identity

- ❌ No queryable state
- ❌ No lifecycle
- ❌ No execution model
- ❌ No memory

As a result, CX remained observation without operation.

II. The Three Architectural Breakthroughs

Three architectural decisions changed everything. Each felt necessary when solving the actual problem, but each represented a categorical departure from existing practice.

Decision 1: Extract Full Context Per Comment (Not Just Driver + Sentiment)

Everyone else:

```

Comment: "The ED wait was terrible, no one told us what was happening"

↓

Output: Driver: "Wait Times" | Sentiment: Negative | Count: 1

```

The New Approach:

```

Comment: "The ED wait was terrible, no one told us what was happening"

↓

Unpacked Mentions:

#### Mention 1:

- Experience Driver: Emergency Services → Triage Efficiency
- Context: "Nearly two-hour wait before any assessment in the ED"
- Emotion: Anger – Frustrated
- Journey Stage: ED Triage Assessment
- Interaction Moment: "while awaiting initial triage assessment after arrival"
- Feedback Type: Complaint
- Opportunity Stream: Fix
- Customer Effort Score: 7
- Keywords: ["ED wait", "triage delay", "assessment wait time"]

#### Mention 2:

- Experience Driver: Support Quality → Communication Style
- Context: "No staff provided updates or explained delays during ED wait"

- Emotion: Fear – Anxious
- Journey Stage: ED Waiting Period
- Interaction Moment: "during the two-hour wait with no status updates"
- Feedback Type: Complaint
- Opportunity Stream: Fix
- Customer Effort Score: 6
- Keywords: ["no updates", "lack of communication", "waiting anxiety"]
- ...

## **Why This Matters:**

### **1. Behavioral truth is preserved**

- Not just "wait times are bad"
- But: "Triage assessment delays for elderly patients" AND "Communication gaps during extended ED waits"

### **2. Multiple drivers can emerge from one comment**

- One narrative often contains multiple behavioral realities
- Traditional extraction collapses these into a single theme

### **3. Context enables execution**

- Interaction moment tells you WHERE to intervene
- Journey stage tells you WHEN in the experience
- Emotion tells you HOW URGENT

### **4. Dimensions prevent drift**

- "Communication" is ambiguous
- "Communication Style during ED waiting periods" is specific
- Same driver, different contexts = different interventions

## **The Dimensions (Context Enrichment):**

### **Each mention captures:**

- Experience Driver: Canonical identity (the WHAT)
- Context: Situational reality (the WHY)
- Emotion: Weighting + urgency signal (the INTENSITY)
- Feedback Type: Intent (Complaint, Compliment, Suggestion)
- Journey Stage: Temporal + process location
- Interaction Moment: Micro-context (precise trigger point)
- Opportunity Stream: Strategic response type (Fix, Optimize, Amplify, Innovate)
- Customer Effort Score: Friction magnitude
- Keywords: Semantic markers for clustering

These are not metadata. These are the variables that define behavioral coherence.

Without them, drivers would remain ambiguous labels. With them, drivers gain operational specificity.

## **Decision 2: Build Comprehensive State Structure (SEU, Not Frequency Counts)**

Everyone else:

~ ~ ~

Driver: "Triage Efficiency"

Mentions: 487

Positive: 23

Negative: 464

Net Sentiment: -90.5%

~ ~ ~

## **The New Approach (Structured Experience Unit):**

~ ~ ~

Experience Driver: Emergency Services → Triage Efficiency

### **SEU State:**

Core Metrics:

- ERI (Emotional Resonance Index): -45.2
  - Tier: Negative
- EVI (Emotional Volatility Index): 72.3
  - Tier: High Volatility
- RF (Recency × Frequency): 87.5
  - Urgency Category: Very High Activity

Derived Intelligence:

- Emotional\_State\_Band: ES4\_Active\_Crisis
- Priority\_Tier: P0
- Priority\_Purpose: "Emergency Crisis Response"

Temporal Intelligence:

Emotion\_Recency\_Profile:

by\_tier:

Tier\_1 (most recent):

emotions: {Anger: 45%, Agitation: 30%, Fear: 25%}

ERI: -52, EVI: 78

ES\_Band: ES4\_Active\_Crisis

Tier\_5 (oldest):

emotions: {Agitation: 60%, Ambivalence: 30%, Appreciation: 10%}

ERI: -18, EVI: 42

ES\_Band: ES3\_At\_Risk

ERI\_Flow:

start: -18 (early period)

end: -52 (recent period)

delta: -34

trend: "↓↓ Strongly Declining"

EVI\_Flow:

start: 42 (early period)

end: 78 (recent period)

delta: +36

trend: "↑↑ Rapidly Fragmenting"

Pattern\_Flow:

pattern\_label: "CRISIS PATTERN: Sentiment collapsing while emotions fragmenting"

PEM\_Guidance:

mode: CRISIS

urgency\_band: Immediate

recommended\_stream\_bias: ["Fix", "Optimize"]

emotional\_driver: "Anger"

Metadata:

- Total\_Mentions: 487

- Most\_Recent\_Date: 2024-12-20

- Age\_Days: 5

- First\_Seen\_Date: 2024-09-15

...

## Why This Matters:

### 1. Queryable State

- Agents can ask: "What's the current condition of Triage Efficiency?"
- Answer is deterministic, not inferred

### 2. Multi-Dimensional

- Not just polarity (ERI)
- But also volatility (EVI) and urgency (RF)
- Plus temporal trajectory (getting worse vs. stable vs. improving)

### 3. Temporal Intelligence

- System knows this is DETERIORATING, not just "bad"
- Can detect pattern: crisis emerging, recovery underway, stable loyalty, etc.

#### 4. Operationally Complete

- Contains everything needed to decide what to do
- No external re-interpretation required
- Priority tier + purpose derived from state

#### The SEU as Primitive:

A Structured Experience Unit (SEU) is the atomic unit of customer experience because:

##### ✓ Atomic: Cannot be subdivided without losing executability

- Remove the driver → lose identity (what are you measuring?)
- Remove emotional weighting → lose signal quality
- Remove temporal grounding → lose urgency context
- Remove the analysis window → lose temporal coherence

##### ✓ Composable: Forms higher-order constructs

- SEUs → Orchestration Units (behavioral missions)
- SEU collections → Portfolios (cross-driver prioritization)
- SEU + temporal intelligence → PDAM loops (autonomous execution)
- SEU lineage → Institutional Memory (learning substrate)

##### ✓ Semantically Stable: Means the same thing across contexts

- Same identity across channels (email, chat, survey)
- Same definition across time windows
- Same interpretation across agents
- Same meaning across organizations

##### ✓ Operationally Complete: Sufficient to execute work

- Determines priority (P0-P5)
- Determines purpose ("Emergency Crisis Response")
- Determines stream bias (Fix vs. Optimize)
- Provides evidence (mention IDs)
- Links to historical outcomes (what worked before)

No other unit in CX satisfies all four criteria.

#### Decision 3: Birth Atomic Missions from Granular Evidence (OUs from Mentions, Not SEUs)

Everyone else:

...

SEU State: "Triage Efficiency is bad (487 mentions, -90.5% sentiment)"

↓

Recommendation: "Improve triage efficiency" ❌

...



(Too vague, not actionable, no behavioral specificity)

The New Approach:

...

SEU State: "Triage Efficiency → ERI -45, EVI 72, Priority P0"

↓

Cluster 487 mentions by behavioral signature:

(Experience Driver × Feedback Type × Opportunity Stream × Behavioral Context)

↓

Generated Orchestration Units:

OU 1: "Elderly Fall Triage Delays – Complaint – Fix"

Behavioral Truth:

- Elderly patients with falls wait 2+ hours for initial assessment

Interaction Moment:

- "awaiting initial triage after elderly fall presentation"

Matters:

- "Delayed triage for elderly falls increases clinical risk of deterioration and undermines trust in emergency access"

Evidence:

- 143 mentions
- Mention IDs: [m\_1847, m\_2910, m\_3384, ...]

Stream Justification:

- "This is a broken emergency access process with direct clinical risk; must be fixed immediately"

Semantic Action Statement:

SECTION 1: THE CUSTOMER REALITY

The customer submitted a Complaint that while awaiting initial triage after arriving at the ED for an elderly fall, no assessment occurred for nearly two hours.

SECTION 2: THE STRATEGIC RESPONSE

We need to Fix → Emergency Services → Triage Efficiency to address delayed initial assessment that jeopardizes timely care and undermines trust in emergency access.

OU 2: "Trauma Protocol Activation Delays – Complaint – Fix"

Behavioral Truth:

- High-acuity trauma cases not triaged fast enough per protocol

Interaction Moment:

- "during acute trauma presentation requiring immediate intervention"

Matters:

- "Life-threatening delays in trauma protocol activation violate clinical standards and risk patient outcomes"

Evidence:

- 87 mentions
- Mention IDs: [m\_982, m\_1653, m\_2477, ...]

Stream Justification:

- "Patient safety emergency requiring immediate protocol review and enforcement"

OU 3: "Minor Injury Wait Time Communication – Complaint – Optimize"

Behavioral Truth:

- Low-acuity cases wait 4+ hours without status updates or expectations

Interaction Moment:

- "during extended wait for non-urgent laceration treatment"

Matters:

- "Poor communication during long waits for minor injuries drives ED abandonment and increases frustration"

Evidence:

- 98 mentions
- Mention IDs: [m\_445, m\_1290, m\_2856, ...]

Stream Justification:

- "Process optimization opportunity to reduce abandonment and improve perceived service quality"

...

Why This Matters:

1. Atomic Missions

- Each OU is a specific, executable action
- Not "improve triage" but "fix elderly fall triage delays"
- Single behavioral truth, single interaction moment

2. Behaviorally Distinct

- Elderly falls ≠ trauma protocol ≠ minor injuries
- Same parent driver, different missions

- Different contexts demand different interventions

### 3. Evidence-Backed

- Mention IDs link back to source customer voices
- Can retrieve original context for any OU
- Verifiable, auditable, traceable

### 4. Agent-Executable

- No interpretation needed
- Just activate the OU
- PDCA cycle can execute directly

The Critical Insight:

If you generate OUs from the aggregated SEU, you lose the behavioral granularity that makes them executable.

Aggregation necessarily loses the specificity required for action.

The only way to preserve behavioral truth is to:

1. Cluster mentions before full aggregation
2. Generate OUs from behavioral clusters
3. Link OUs to parent SEU
4. Preserve evidence lineage

This is the same pattern as:

- Git: Diffs (mentions) → Commits (OUs) → Branch state (SEU)
- Bitcoin: Individual transactions (mentions) → Transaction outputs (OUs) → UTXO set (SEU)
- Double-entry bookkeeping: Journal entries (mentions) → Transaction batches (OUs) → Account balance (SEU)

---

## III. The Complete Architecture

### Layer 1: Unpacking (Raw Feedback → Mentions with Dimensions)

Input: Unstructured customer feedback from disparate sources

- Surveys, support tickets, reviews, social media, emails, chat logs

Process: Behavioral decomposition via LLM

- Each comment is unpacked into 1:M experience driver mentions
- Each mention captures full dimensional context
- Same driver can appear multiple times in one comment

- Same driver can appear across many comments

Output: Granular mention rows with preserved context

Example Scale:

- 100,000 raw comments
- → 250,000 experience driver mention rows
- Each mention fully dimensional

Key Innovation:

- Traditional systems extract ONE theme per comment
- This architecture extracts ALL behavioral truths per comment
- Context is preserved, not collapsed

## **Layer 2: State Aggregation (Mentions → SEUs)**

Input: 250k mention rows (from Layer 1)

Process: Aggregate by Experience Driver per Analysis Window

```
```python
```

```
grouped = mentions.groupby("experience_driver")
```

```
for driver, mentions_group in grouped:
```

```
    Compute emotional state
```

```
    ERI = compute_emotional_resonance(mentions_group)
```

```
    EVI = compute_emotional_volatility(mentions_group)
```

```
    Compute urgency
```

```
    RF = compute_recency_frequency(mentions_group)
```

```
    Compute temporal intelligence
```

```
    temporal_intel = compute_temporal_flows(mentions_group)
```

```
    Derive strategic intelligence
```

```
    emotional_state_band = classify_state(ERI, EVI)
```

```
    priority_tier = derive_priority(emotional_state_band, RF)
```

```
    Create SEU
```

```
    seu = StructuredExperienceUnit(
```

```
        experience_driver=driver,
```

```
        ERI=ERI,
```

```
        EVI=EVI,
```

```
        RF=RF,
```

```
        temporal_intelligence=temporal_intel,
```

```
        emotional_state_band=emotional_state_band,
```

```

        priority_tier=priority_tier,
        mentions=mentions_group
    )
    ...

```

Output: N SEUs (one per unique Experience Driver in the window)

Key Constraint (Identity Law):

```

```sql
UNIQUE(experience_driver_id, window_id)
```

```

One driver → One canonical SEU per analysis window

The SEU Structure:

```

...
SEU = {
    identity: Experience Driver (canonical)
    state: {
        ERI: emotional polarity (-100 to +100)
        EVI: emotional volatility (0 to 100)
        RF: recency × frequency urgency (0 to 100)
    }
    temporal_intelligence: {
        ERI_Flow: trajectory of polarity over time
        EVI_Flow: trajectory of volatility over time
        ES_Flow: migration across emotional state bands
        Emotion_Flow: composition shifts (which emotions rising/falling)
        Pattern_Flow: interpreted temporal pattern
        Context_Flow: narrative summary
        PEM_Guidance: derived guidance (mode, urgency, stream bias)
    }
    derived_state: {
        Emotional_State_Band: ES1..ES5
        Priority_Tier: P0..P5
        Priority_Purpose: strategic interpretation
    }
    metadata: {
        total_mentions: count
        first_seen_date: datetime
        most_recent_date: datetime
        age_days: int
    }
}
...

```

Key Innovation:

- Not just counts and averages
- Multi-dimensional state (polarity + volatility + urgency)
- Temporal intelligence (trajectory, not just snapshot)
- Deterministic (same inputs → same outputs)
- Queryable (SQL, not LLM inference)

Layer 2.5: Mission Generation (Mentions → OUs via Behavioral Clustering)

Input: Mentions under a single SEU

Process: Behavioral clustering

```
```python
```

```
For each SEU, cluster its mentions by behavioral signature
for seu in seus:
```

```
 mentions = seu.get_mentions()
```

```
 Cluster by:
```

- Experience Driver (fixed - all mentions share this)
- Feedback Type (Complaint vs Compliment vs Suggestion)
- Opportunity Stream (Fix vs Optimize vs Amplify vs Innovate)
- Behavioral Signature (interaction moment + context similarity)

```
 behavioral_clusters = cluster_mentions(
 mentions,
 dimensions=["feedback_type", "opportunity_stream", "interaction_moment"]
)
```

```
 ous = []
```

```
 for cluster in behavioral_clusters:
```

```
 ou = OrchestrationUnit(
 seu_id=seu.id,
 behavioral_truth=extract_behavioral_truth(cluster),
 interaction_moment=extract_interaction_moment(cluster),
 matters=extract_matters(cluster),
 evidence_count=len(cluster),
 evidence_mention_ids=[m.id for m in cluster],
 semantic_action_statement=generate_action_statement(cluster)
)
```

```
 ous.append(ou)
```

```
    ```
```

Output: M Orchestration Units per SEU

Cardinality:

...

1 SEU → M OUs (behavioral missions)

...

Key Innovation:

- OUs are generated from mentions, NOT from the aggregated SEU
- Preserves behavioral granularity
- Each OU is atomic and executable
- Evidence lineage is maintained (mention IDs)

Why This Order Matters:

 Wrong: Aggregate first → Generate OUs from SEU

- Loses behavioral specificity
- Results in vague missions

 Correct: Cluster first → Aggregate state separately

- Preserves behavioral truth
- OUs remain atomic
- SEU remains queryable

Layer 3: Orchestration (OU Activation → PDCA → Outcomes → Memory)

Input: Orchestration Units (ready for activation)

Process: PDAM Loop (Perceive → Decide → Act → Memory)

...

1. PERCEIVE

- Query SEU state
- Identify priority OUs
- Check historical outcomes for similar patterns

2. DECIDE

- Select OU for activation based on:
 - * Priority tier
 - * Historical success rate
 - * Resource availability
 - * Strategic alignment

3. ACT

- Activate OU
- Execute PDCA cycle:

- * Plan: Define intervention
- * Do: Execute intervention
- * Check: Measure before/after SEU state
- * Act: Standardize if successful, adjust if not

4. MEMORY

- Record OU activation
 - Store outcome evaluation (SEU delta)
 - Update institutional memory
 - Inform future orchestration decisions
- ````

The PDCA Cycle:

```

````sql
-- Plan
INSERT INTO problem_statements (ou_activation_id, statement)
VALUES (...);

-- Do
INSERT INTO pdca_cycles (problem_statement_id, plan, do_action)
VALUES (...);

-- Check
INSERT INTO outcome_evaluations (
 pdca_cycle_id,
 seu_state_before,
 seu_state_after,
 eri_delta,
 evi_delta,
 outcome_classification
) VALUES (...);

-- Act (Memory)
INSERT INTO institutional_memory_events (
 source_type,
 source_id,
 event_type,
 outcome,
 learnings
) VALUES (...);
````

```

Key Innovation:

- Outcomes are verifiable (before/after SEU comparison)

- Memory persists (future activations use past outcomes)
- Learning compounds (system gets smarter over time)
- Causality is traceable (action → outcome linkage)

The Emotional Relational Model (ERM) as Enforcement Substrate

The ERM is not the primitive. It is the relational substrate that enforces the laws under which SEUs can exist, evolve, and retain memory.

The Three Laws:

1. Identity Law

```

Each Experience Driver has exactly one canonical SEU per analysis window.

Enforcement:

UNIQUE(experience\_driver\_id, window\_id)

Result:

- No duplicate states
- No parallel interpretations
- Single source of truth per driver per timeframe

```

2. Lineage Law

```

Every OU activation is a timestamped state transition linked to a SEU, carrying outcome and consequence.

Enforcement:

FK(orchestration\_unit\_id) → orchestration\_units(id)

FK(seu\_id) → seus(id)

immutable activation records

Result:

- Every action is tied to the driver state that triggered it
- Every outcome traces back to a specific condition
- Causal chain is preserved

```

3. Memory Law

```

Every Experience Driver state transition retains referential integrity

across time.

Enforcement:

- FK relationships across all state transitions
- institutional\_memory\_events table
- full lineage preservation

Result:

- No driver state is ever orphaned
- Historical outcomes constrain future decisions
- System can query its own biography

```\n

The Schema (Core Entities):

```\n

experience\_drivers

- Canonical list of drivers
- UNIQUE(name) or UNIQUE(external\_id)

seus (Structured Experience Units)

- Experience state account per driver per window
- FK(experience\_driver\_id) → experience\_drivers(id)
- UNIQUE(experience\_driver\_id, window\_id)

mentions

- Raw unpacked mentions with full dimensions
- FK(experience\_driver\_id) → experience\_drivers(id)
- FK(seu\_id) → seus(id)

orchestration\_units

- Mission templates derived from behavioral clusters
- FK(seu\_id) → seus(id)

ou\_activations

- Executed instances of an OU
- FK(orchestration\_unit\_id) → orchestration\_units(id)
- UNIQUE(orchestration\_unit\_id, activated\_at)

problem\_statements

- PDCA-ready problem articulation
- FK(ou\_activation\_id) → ou\_activations(id)

pdca\_cycles

- Plan–Do–Check–Act iterations

- FK(problem\_statement\_id) → problem\_statements(id)

outcome\_evaluations

- Before/after SEU state comparison
- FK(pdca\_cycle\_id) → pdca\_cycles(id)

institutional\_memory\_events

- All lineage events (activations, outcomes, patterns)
- FK(source\_entity\_id, source\_type)

...

### **Properties That Emerge from the Laws:**

- ✓ Full auditability: Instant root cause analysis via lineage
- ✓ Agent-native state: Agents query shared truth, no hallucination
- ✓ Elasticity verification: Provable action → outcome causality
- ✓ Institutional memory: System remembers patterns humans forget
- ✓ BI compatibility: Standard SQL queries (Tableau, Power BI)
- ✓ Model independence: Swap LLMs without rewriting state
- ✓ Horizontal scalability: Billions of rows without redesign
- ✓ Zero lock-in: Portable SQL schema, organization owns the ledger

---

## **IV. Why These Decisions Enable Everything Else**

The three architectural decisions—full context extraction, comprehensive state structures, and behavioral mission generation—are not arbitrary. They are the minimum necessary conditions for experience drivers to become computable.

What follows are not additional features. They are emergent properties of the architecture.

### **Agentic Readiness (Emergent Property)**

#### **Why agents fail on traditional CX systems:**

- No shared truth (every agent re-infers driver state)
- State is inferred repeatedly (hallucination risk)
- Memory is shallow (context window limited)
- Intent is implicit (vague recommendations)
- Actions are unconstrained (no guardrails)

#### **Why agents work on this architecture:**

### ✓ Canonical Entities

```
```python
agent.query("What is the current state of Triage Efficiency?")
→ Returns: SEU with ERI, EVI, RF, Priority, Temporal Intelligence
→ Deterministic (same query, same result, always)
```
```

### ✓ Authoritative State

```
```python
agent.query("Show me the trajectory of Triage Efficiency over 60 days")
→ Returns: ERI_Flow, EVI_Flow, Pattern_Flow
→ No inference, just retrieval
```
```

### ✓ Allowed Transitions

```
```python
agent.query("What are the executable missions for Triage Efficiency?")
→ Returns: List of OUs with evidence counts, interaction moments, matters
→ Agent can activate, not invent
```
```

### ✓ Irreversible Memory

```
```python
agent.query("What interventions have worked historically for elderly fall triage delays?")
→ Returns: Previous OU activations with outcomes
→ Agent learns from institutional memory
```
```

### ✓ Queryable Reality

```
```sql
-- Agent executes deterministic SQL, not probabilistic LLM generation
SELECT
    seu.experience_driver,
    seu.ERI,
    seu.EVI,
    seu.priority_tier,
    COUNT(ou.id) as pending_missions
FROM seus seu
LEFT JOIN orchestration_units ou ON ou.seu_id = seu.id
WHERE seu.priority_tier IN ('P0', 'P1')
AND ou.activated = false
GROUP BY seu.id
ORDER BY seu.RF DESC;
```
```

Result: Agents operate inside shared truth, not parallel interpretations.

## Institutional Memory (Emergent Property)

### What memory requires:

- Persistent identity (same driver across time)
- State lineage (how did we get here?)
- Outcome records (what happened when we acted?)
- Referential integrity (no orphaned history)

### How this architecture provides it:

#### ✓ Persistent Identity

- Experience Drivers are canonical
- `emergency\_services\_triage\_efficiency` is the same entity in January and June
- No drift, no renaming, no loss of continuity

#### ✓ State Lineage

```
```sql
-- Query: How has Triage Efficiency evolved?
SELECT
    window_start_date,
    ERI,
    EVI,
    priority_tier,
    emotional_state_band
FROM seus
WHERE experience_driver_id = 'emergency_services_triage_efficiency'
ORDER BY window_start_date ASC;
```
```

#### ✓ Outcome Records

```
```sql
-- Query: What interventions have we tried for this driver?
SELECT
    ou.name,
    ou_act.activated_at,
    eval.eri_delta,
    eval.outcome_classification
FROM orchestration_units ou
JOIN ou_activations ou_act ON ou_act.orchestration_unit_id = ou.id
JOIN outcome_evaluations eval ON eval.ou_activation_id = ou_act.id
WHERE ou.seu_id IN (
```

```

    SELECT id FROM seus WHERE experience_driver_id =
'emergency_services_triage_efficiency'
)
ORDER BY ou_act.activated_at DESC;
```

```

### ✓ Pattern Recognition

```

```sql
-- Query: Which types of interventions historically work best for crisis-state drivers?
SELECT
    ou.opportunity_stream,
    AVG(eval.eri_delta) as avg_improvement,
    COUNT(*) as activation_count
FROM orchestration_units ou
JOIN ou_activations ou_act ON ou_act.orchestration_unit_id = ou.id
JOIN outcome_evaluations eval ON eval.ou_activation_id = ou_act.id
JOIN seus seu ON seu.id = ou.seu_id
WHERE seu.emotional_state_band = 'ES4_Active_Crisis'
AND eval.outcome_classification = 'Successful'
GROUP BY ou.opportunity_stream
ORDER BY avg_improvement DESC;
```

```

Result: The system accumulates organizational knowledge. Past successes constrain future decisions. Failures are not repeated.

## Outcome Evaluation (Emergent Property)

### What outcome evaluation requires:

- Baseline state (what was the condition before?)
- Intervention record (what did we do?)
- Result state (what changed after?)
- Causal linkage (can we prove the intervention caused the change?)

### How this architecture provides it:

#### ✓ Before State

```

```python

```

Capture SEU state before OU activation

```
seu_before = query_seu_state(  
    experience_driver='emergency_services_triage_efficiency',  
    window_end_date='2024-11-30'  
)
```

Returns: ERI=-45, EVI=72, RF=87, Priority=P0

```

### ✓ Intervention Record

```
```python
```

Activate OU

```
activation = activate_orchestration_unit(  
    ou_id='elderly_fall_triage_delays_fix',  
    activated_at='2024-12-01',  
    intervention_description='Added dedicated triage nurse for elderly patients'  
)
```

```

### ✓ After State

```
```python
```

Measure SEU state after intervention period

```
seu_after = query_seu_state(  
    experience_driver='emergency_services_triage_efficiency',  
    window_end_date='2024-12-31'  
)
```

Returns: ERI=-27, EVI=58, RF=65, Priority=P2

```

## ✓ Causal Linkage

```
```python
```

Record outcome evaluation

```
outcome = evaluate_outcome(  
    ou_activation_id=activation.id,  
    seu_before=seu_before,  
    seu_after=seu_after,  
    delta_eri=+18, Improved from -45 to -27  
    delta_evi=-14, Reduced from 72 to 58 (consolidating)  
    delta_rf=-22, Reduced from 87 to 65 (less urgent)  
    outcome_classification='Successful',  
    notes='Elderly fall triage time reduced from avg 120min to 45min'  
)  
```
```

### The Proof Mechanism:

```
```sql
```

-- Query: Did this intervention work?

```
SELECT  
    ou.name,  
    ou_act.activated_at,  
    eval.eri_before,  
    eval.eri_after,  
    eval.eri_delta,  
    eval.evi_before,  
    eval.evi_after,
```



```

    eval.outcome_classification

FROM outcome_evaluations eval

JOIN ou_activations ou_act ON ou_act.id = eval.ou_activation_id

JOIN orchestration_units ou ON ou.id = ou_act.orchestration_unit_id

WHERE ou.name = 'Elderly Fall Triage Delays – Complaint – Fix'

ORDER BY ou_act.activated_at DESC

LIMIT 1;

```

```

Result: Verifiable proof of causality. Actions that work can be repeated. Actions that fail can be adjusted or abandoned.

## Compound Learning (Emergent Property)

### What compound learning requires:

- Memory that persists (not reset each period)
- Outcomes that inform decisions (priors, not just observations)
- Patterns that generalize (learning transfers across contexts)
- Continuous improvement (system gets smarter over time)

### How this architecture enables it:

#### ✓ Memory Persistence

- Institutional memory events never expire
- Historical outcomes remain queryable indefinitely
- Each new window builds on previous knowledge

#### ✓ Outcome-Informed Decisions

```
```python
```

When selecting which OU to activate next, query historical success rates

```
prioritized_ous = query_ous_with_historical_success(
```

```

    seu_id='emergency_services_triage_efficiency',

    priority_tiers=['P0', 'P1'],

    order_by='historical_success_rate DESC'

)

```

Returns OUs ranked by:

- How often this type of intervention worked before
- Average improvement (ERI delta)
- Time to impact
- Resource efficiency

```

### ✓ Pattern Generalization

```

```python

```

Learn: "Crisis-state drivers with high volatility respond better to Fix interventions than Optimize"

```

pattern = query_intervention_effectiveness(

    emotional_state_band='ES4_Active_Crisis',

    evi_range=[60, 100],

    opportunity_streams=['Fix', 'Optimize']

)

```

Apply learning: Bias future activations toward 'Fix' for similar conditions

```

### ✓ Continuous Improvement

```

Cycle 1: Activate OU → Measure outcome → Record learning

Cycle 2: Use Cycle 1 outcome as prior → Activate next OU → Measure → Record

Cycle 3: Use Cycles 1+2 outcomes → Select optimal intervention → Execute

...

Cycle N: System has accumulated N learnings, each activation is better informed

...

The Feedback Loop:

...

OU Activation

↓

Outcome Measurement (SEU delta)

↓

Memory Storage (institutional_memory_events)

↓

Pattern Recognition (which interventions work in which contexts?)

↓

Informed Orchestration (use patterns to select next OU)

↓

Better Outcomes (learning compounds)

↓

[Loop repeats, system improves continuously]

...

Result: Learning accumulates. The system doesn't just execute—it optimizes execution over time based on accumulated evidence.

V. The Inevitability Thesis

This architecture is not merely useful. It is inevitable.

Why the Driver Must Remain Center Stage

The driver is the organizational unit of accountability.

CX teams don't organize around:

- ❌ Mentions (too granular)
- ❌ OUs (too tactical)
- ❌ Emotions (too volatile)
- ❌ Contexts (too fragmented)

They organize around drivers:

- "Who owns the Delivery Speed driver?"
- "What's the quarterly goal for Triage Efficiency?"
- "How much budget do we allocate to Provider Communication?"

The driver is the unit of strategic memory.

Executives don't ask:

- ❌ "What were the top themes this quarter?"

They ask:

- ✅ "Is Refund Cycle Time improving?"
- ✅ "Has Call Center Wait Time stabilized?"
- ✅ "Are we still losing customers over Product Availability?"

The driver is the coordination primitive.

Multiple teams, multiple agents, multiple systems need a shared reference:

...

Agent 1 (Perceive): "I detected a spike in Triage Efficiency complaints"

Agent 2 (Decide): "Query the SEU for Triage Efficiency. ERI = -45, Priority = P0"

Agent 3 (Act): "Activate OU: Elderly fall triage delays"

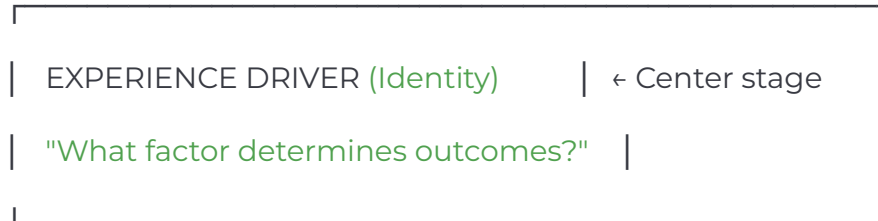
Agent 4 (Memory): "Log outcome: Triage time reduced 120min → 45min, ERI improved +18"

...

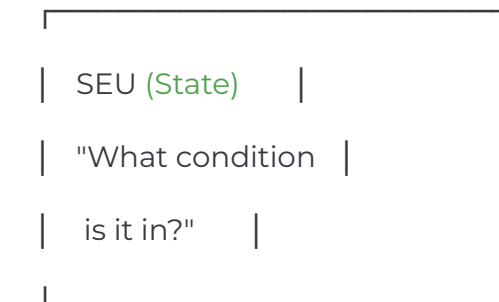
The driver is the entity they all coordinate around.

The Correct Hierarchy

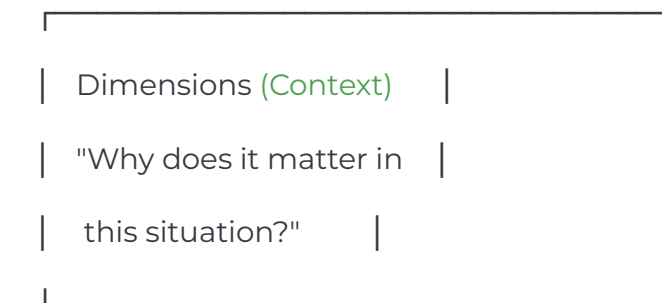
...



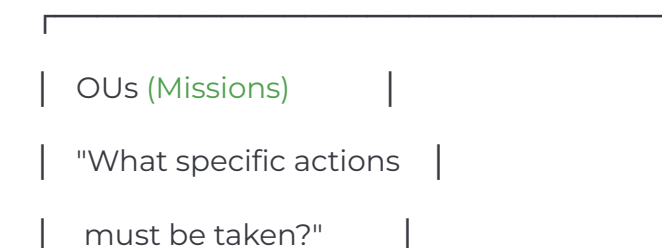
↓



↓



↓



...

The driver is the root. Everything else branches from it.

This hierarchy cannot be inverted:

- ❌ OUs cannot exist without drivers (what are they acting on?)
- ❌ Dimensions cannot exist without drivers (context for what?)
- ❌ SEUs cannot exist without drivers (state of what?)

The driver must remain center stage because it is the only entity that:

- Has organizational meaning (teams own drivers, not mentions)
- Has strategic continuity (executives track drivers, not OUs)
- Has computational stability (canonical identity doesn't drift)
- Has cross-domain generalizability (drivers exist in all experience contexts)

Why This Couldn't Have Been Built Before Now

Three enabling conditions had to converge:

1. LLMs for Context Extraction (2022+)

Before large language models:

- No way to reliably extract Experience Drivers from unstructured feedback at scale
- No way to unpack behavioral context accurately
- No way to identify interaction moments, journey stages, or emotional nuance

Why it matters:

- Traditional NLP: keyword matching, topic modeling (drift, inaccuracy)
- LLMs: semantic understanding, context preservation, multi-dimensional extraction

The breakthrough:

- GPT-3.5+ can unpack a single comment into multiple behaviorally distinct mentions
- Can extract canonical driver identity + full contextual dimensions
- Can do this at scale (millions of comments)

Without LLMs, Layer 1 (unpacking) was impossible at enterprise scale.

2. Agentic Systems Creating Demand (2023+)

Before agentic AI:

- Humans mediated all CX action
- No need for machine-queryable state
- Recommendations were consumed by humans, who added context and interpretation

Why it matters:

- Agents cannot hallucinate driver state and act safely
- Agents need deterministic, shared truth
- Agents need atomic, executable missions

The breakthrough:

- Organizations now want autonomous CX systems
- But agents fail without persistent state and memory
- The demand for the Experience State Layer emerged only when agents became viable

Without agents, there was no urgency to build this substrate.

3. Infrastructure Maturity (Cloud, Postgres, Modern Stacks)

Before cloud infrastructure maturity:

- No cost-effective way to maintain lineage at scale

- No easy deployment of relational schemas
- No standard tooling for billion-row datasets

Why it matters:

- This architecture requires persistent state across billions of mentions
- Requires complex joins, temporal queries, referential integrity
- Requires lineage preservation (every activation, every outcome)

The breakthrough:

- Modern Postgres handles this scale easily
- Cloud deployment makes it accessible
- Docker/K8s make it portable

Without infrastructure maturity, the ERM would have been prohibitively expensive.

Why This Is Inevitable Going Forward

Every customer-facing organization will eventually need this because:

1. Agentic CX Is the Future

- Autonomous orchestration is not optional—it's the next evolution
- But agents cannot operate on themes, sentiments, or dashboards
- They need the Experience State Layer

2. Memory Is Competitive Advantage

- Organizations that remember what worked will outperform those that don't
- Institutional memory compounds over time
- Without persistent driver state, memory is impossible

3. Governance Will Be Required

- Regulators will demand auditability of CX decisions

- "Why did you prioritize X over Y?" must have a traceable answer
- The ERM provides full lineage (action → outcome → learning)

4. Scale Demands Infrastructure

- Manual CX analysis doesn't scale to millions of customers
- Vague recommendations don't scale to hundreds of teams
- Only infrastructure—persistent state, atomic missions, compound learning—can scale

5. The Driver Has Always Been the Right Unit

- CX practitioners have organized around drivers for 20+ years
- They were right, but lacked the computational substrate
- This architecture redeems the driver, doesn't replace it

Result:

- Early adopters will gain compound learning advantage
- Late adopters will be forced to adopt as agents become table-stakes
- The Experience State Layer will become invisible infrastructure (like databases, like Git)

VI. The Category of Innovation

Not "Better CX Analytics"

This is not:

- ✗ Faster sentiment analysis
- ✗ More accurate theme extraction
- ✗ Better dashboards
- ✗ AI-powered insights

Those are product innovations and improvements to existing solutions. This is infrastructure innovation - a missing layer beneath all products. Not "AI-Powered Insights"

LLMs are used in Layer 1 (unpacking), but they are not the innovation.

The innovation is:

- ✓ Making drivers persistent entities (not ephemeral themes)
- ✓ Giving drivers queryable state (not just mention counts)
- ✓ Generating atomic missions from behavioral clusters (not from aggregated summaries)
- ✓ Enforcing lineage and memory via relational constraints (not via LLM recall)

LLMs enable the architecture. They are not the architecture.

Foundational Infrastructure (The Missing Computational Substrate)

This work sits in the same category as:

| Innovation | What It Did | Result |

|-----|-----|-----|

| Codd's Relational Model (1970) | Gave data persistent structure | Entire software industry reorganized around databases |

| Nakamoto's Bitcoin (2008) | Gave value persistent state without trusted intermediaries | Entire blockchain ecosystem emerged |

| Torvalds' Git (2005) | Gave code collaboration persistent lineage | Entire open-source ecosystem reorganized |

| This Work (2025) | Gave experience drivers persistent state | CX industry will reorganize around driver infrastructure |

The pattern:

1. A domain that matters lacks computational substrate

2. Someone formalizes the primitive + enforces invariants
3. Infrastructure becomes invisible but essential
4. Entire ecosystems emerge on top of it

The Codd Comparison (Most Direct Analogy)

Edgar Codd (1970): Relational Model for Data

Problem:

- Data existed in application-specific formats
- Every app had custom query logic
- No standard structure, no portability, no shared truth

Solution:

- Define tables, rows, columns (primitives)
- Enforce constraints (foreign keys, uniqueness, referential integrity)
- Enable declarative queries (SQL)

Result:

- Data became queryable
- Applications became composable
- Entire software industry reorganized around relational databases

This Work (2025): Relational Model for Experience Drivers

Problem:

- Experience drivers existed as analyst interpretations
- Every analysis started from scratch
- No standard structure, no continuity, no shared truth

Solution:

- Define Experience Drivers, SEUs, OUs (primitives)
- Enforce constraints (Identity Law, Lineage Law, Memory Law)
- Enable declarative queries (SQL on driver state)

Result:

- Experience drivers become queryable
- CX systems become composable
- CX industry will reorganize around driver infrastructure

Both are infrastructure work. Both formalize what was previously informal. Both enable computation that was previously impossible.

Why It Feels Obvious in Retrospect (Signature of Foundational Work)**When Codd proposed the relational model:**

- IBM ignored it (they had IMS, a hierarchical database)
- Academics said "interesting but impractical"
- It took 10 years before Oracle made it commercial

In retrospect:

- "Of course data should have standard structure"
- "Of course we should be able to query it declaratively"
- "Why did we ever do it differently?"

When this work is explained to CX leaders:

- They say: "Wait, drivers don't have persistent state?"
- They say: "They reset every quarter?"
- They say: "You can't query the current condition of a driver?"
- They say: "Why the hell not?"

In retrospect (5 years from now):

- "Of course drivers should have canonical identity"
- "Of course we should be able to query driver state"
- "Why did we ever do it differently?"

VII. What This Actually Is

The Experience State Layer

Definition:

The Experience State Layer is the missing computational layer between human expression and machine operation. It holds the reasons behind human experience as authoritative system state.

What it does:

- Instantiates experience drivers as persistent entities
- Maintains queryable state for each driver
- Enables deterministic reasoning about experience
- Provides memory and lineage for all interventions

What it replaces:

- Thematic analysis (replaced with canonical drivers)
- Sentiment dashboards (replaced with multi-dimensional state)
- Generic recommendations (replaced with atomic missions)
- Episodic learning (replaced with compound memory)

What it enables:

- Agentic orchestration (safe autonomous action)
- Institutional memory (what worked, what didn't)
- Outcome verification (provable causality)

- Continuous improvement (learning compounds)

The Emotional Relational Model (ERM)

Definition:

The ERM is the relational enforcement layer that makes Experience Drivers computable, governable, and institutional. It is the ground-truth ledger of Experience Driver state.

The Three Laws:

Identity Law: Each Experience Driver has exactly one canonical SEU per analysis window.

Lineage Law: Every OU activation is a timestamped state transition linked to a SEU, recording intervention, context, and outcome.

Memory Law: Every Experience Driver state transition retains referential integrity across time. No state is ever orphaned.

What it enforces:

- Canonical identity (no duplicates, no drift)
- Temporal continuity (state evolves, never resets)
- Causal lineage (every action traces to trigger and outcome)
- Referential integrity (all relationships maintained)

What it provides:

- Governability (audit trails, compliance)
- Predictability (deterministic queries)
- Reliability (agent-safe operations)
- Scalability (standard relational operations)

The Primitives

Experience Driver (Identity)

- Canonical name: `Category → Subcategory`
- Example: `Emergency Services → Triage Efficiency`
- Represents: The underlying determinant of customer outcomes
- Properties: Stable, system-level, operational surface

Structured Experience Unit (SEU) (State)

- The authoritative state container for an Experience Driver over a defined analysis window
- Contains: ERI, EVI, RF, Temporal Intelligence, Priority, Purpose
- Represents: Current condition of the driver, emotionally weighted and temporally grounded
- Properties: Queryable, deterministic, operationally complete

Orchestration Unit (OU) (Mission)

- Atomic behavioral mission derived from clustering mentions under an SEU
- Contains: Behavioral truth, interaction moment, matters, evidence, action statement
- Represents: Specific executable intervention for a behavioral cluster
- Properties: Atomic, evidence-backed, agent-executable

Dimensions (Context)

- Full behavioral context per mention
- Contains: Journey stage, interaction moment, feedback type, emotion, opportunity stream, CES, keywords
- Represents: The "why" behind the "what" (driver + context)
- Properties: Preserves behavioral truth, enables clustering, prevents drift

The Outcome

Before this work:

- Experience drivers were concepts
- CX was observation
- Insights were outputs
- Memory was human
- Actions were vague

After this work:

- Experience drivers are entities
- CX is computation
- Drivers are inputs
- Memory is system
- Actions are atomic

The categorical shift:

Experience drivers enter computation. Not as text. Not as insights. As state. Once they do, systems can finally act, not just observe.

Appendices

Appendix A: Healthcare Unpacking Example (Full)

Raw Customer Feedback:

"I brought my mother to the emergency department last night after a fall. We waited almost two hours before anyone came to assess her, and during that time, no one explained what was happening. When she was finally seen, the doctor seemed rushed and didn't involve us in the decision about whether to admit her. The ward we were sent to had no working call bell, so she felt unsafe through the night. I honestly don't feel like the hospital respects elderly patients, and I'm worried what would happen if she needed urgent care again. We're even thinking of switching to another hospital if this happens again."

Unpacked Mentions (4 from 1 comment):

Mention 1:

- Experience Driver: Emergency Services → Triage Efficiency
- Entity Name: Emergency Department Triage Assessment
- Theme: Clinical Services & Care Delivery
- Context: "After arriving at the ED for an elderly fall, the family waited nearly two hours before initial assessment, indicating a triage/priority assessment delay."
- Keywords: ["ED wait", "triage delay", "assessment wait time", "elderly fall", "access to care"]
- Feedback Type: Complaint
- Emotion: Anger – Frustrated
- Customer Journey: Emergency Care
- Customer Journey Stage: ED Triage Assessment
- Interaction Moment: "while awaiting initial triage assessment after arrival"
- Opportunity Maximisation Stream: Fix

- Customer Effort Score: 7

- Orchestration Unit Name: "Emergency Department Triage Assessment – Complaint – Fix"

- Semantic Action Statement:

- SECTION 1: THE CUSTOMER REALITY

The customer submitted a Complaint that while awaiting initial triage after arriving at the ED for an elderly fall, no assessment occurred for nearly two hours.

- SECTION 2: THE STRATEGIC RESPONSE

We need to Fix → Emergency Services → Triage Efficiency to address delayed initial assessment that jeopardizes timely care and undermines trust in emergency access.

- Matters: "Delayed triage assessment in the ED slows urgent evaluation for an elderly fall and increases risk of deterioration."

- Stream Justification: "This is a broken emergency access process (triage delay) with direct clinical risk; must be fixed."

- Behavioral Impact: "Improve emergency access speed, reduce clinical risk, and restore trust in ED responsiveness."

Mention 2:

- Experience Driver: Support Quality → Communication Style

- Entity Name: ED Waiting Area Status Updates

- Theme: Patient Support & Care Coordination

- Context: "During the prolonged ED wait, no staff provided updates or explained delays, leaving the elderly patient and family uncertain about next steps."

- Keywords: ["no updates", "lack of communication", "waiting anxiety", "ED staff", "status explanation"]

- Feedback Type: Complaint

- Emotion: Fear – Anxious

- Customer Journey: Emergency Care

- Customer Journey Stage: ED Waiting Period
- Interaction Moment: "during the two-hour wait with no status updates"
- Opportunity Maximisation Stream: Fix
- Customer Effort Score: 6
- Orchestration Unit Name: "ED Waiting Area Status Updates – Complaint – Fix"
- Semantic Action Statement:

- SECTION 1: THE CUSTOMER REALITY

The customer submitted a Complaint that during a prolonged ED wait, no one explained what was happening or provided status updates.

- SECTION 2: THE STRATEGIC RESPONSE

We need to Fix → Support Quality → Communication Style to ensure timely, clear ED status updates that reduce anxiety and guide expectations during waits.

- Matters: "Absence of ED status updates during extended waits increases anxiety and leaves patients uninformed about their care timeline."
- Stream Justification: "Communication during ED waits is a core support function; its absence is a service failure requiring a fix."
- Behavioral Impact: "Reduce distress, improve perceived reliability, and decrease escalations during ED delays."

Mention 3:

- Experience Driver: Consultation Experience → Provider Communication
- Entity Name: Admission Decision Discussion (ED Physician)
- Theme: Clinical Services & Care Delivery
- Context: "During the evaluation, the physician appeared rushed and did not involve the patient/family in the decision regarding admission."
- Keywords: ["rushed consult", "shared decision-making", "admission decision", "elderly patient", "provider communication"]
- Feedback Type: Complaint
- Emotion: Anger – Indignant

- Customer Journey: Clinical Consultation
- Customer Journey Stage: Admission Decision
- Interaction Moment: "during the physician's evaluation about whether to admit"
- Opportunity Maximisation Stream: Fix
- Customer Effort Score: 6
- Orchestration Unit Name: "Admission Decision Discussion (ED Physician) – Complaint – Fix"
- Semantic Action Statement:
- SECTION 1: THE CUSTOMER REALITY

The customer submitted a Complaint that during the physician evaluation, the doctor was rushed and did not include the patient/family in the admission decision.

- SECTION 2: THE STRATEGIC RESPONSE

We need to Fix → Consultation Experience → Provider Communication to ensure shared decision-making and respectful involvement of patients and caregivers in admission choices.

- Matters: "Excluding patient/family from admission decisions diminishes trust and may lead to inappropriate or poorly understood care plans."

- Stream Justification: "Shared decision-making is a core standard; lack of it is a care delivery failure requiring corrective action."

- Behavioral Impact: "Increase respect, comprehension, and adherence by embedding shared decision-making in ED admissions."

Mention 4:

- Experience Driver: Risk Assessment → Operational Risk
- Entity Name: Inpatient Ward Nurse Call Bell System
- Theme: Quality & Patient Safety
- Context: "In the assigned ward, the nurse call bell was nonfunctional overnight, preventing the patient from summoning help and creating a safety risk."

- Keywords: ["call bell failure", "patient safety", "equipment failure", "overnight risk", "ward"]

- Feedback Type: Complaint

- Emotion: Fear – Insecure

- Customer Journey: Inpatient Stay

- Customer Journey Stage: Room Safety & Equipment

- Interaction Moment: "overnight when attempting to request assistance via the call bell"

- Opportunity Maximisation Stream: Fix

- Customer Effort Score: 7

- Orchestration Unit Name: "Inpatient Ward Nurse Call Bell System – Complaint – Fix"

- Semantic Action Statement:

- SECTION 1: THE CUSTOMER REALITY

The customer submitted a Complaint that the ward's nurse call bell did not work overnight, leaving the patient unable to summon help and feeling unsafe.

- SECTION 2: THE STRATEGIC RESPONSE

We need to Fix → Risk Assessment → Operational Risk to restore reliable nurse call functionality that ensures timely assistance and prevents safety incidents.

- Matters: "Nonfunctional nurse call system prevents timely assistance and exposes patients to avoidable harm."

- Stream Justification: "This is a patient safety hazard and equipment reliability failure that requires immediate remediation."

- Behavioral Impact: "Reduce risk of adverse events, improve perceived safety, and reinforce trust in inpatient care."

Key Observations:

- 1 comment → 4 distinct Experience Driver mentions

- Each mention preserves full behavioral context

- Same journey (ED visit) → multiple drivers (triage, communication, consultation, safety)
- Each mention becomes evidence for a distinct OU
- Context dimensions enable behavioral clustering later

Appendix B: Layer 2 Computation Logic (Code Snippets)

Computing ERI (Emotional Resonance Index):

```
```python
```

```
def _compute_eri_from_counts(self, emotion_counts: dict, total_mentions: int) -> float:
```

```
 """
```

```
 Compute ERI from emotion_counts dict.
```

```
 Returns ERI in range [-100, +100].
```

```
 """
```

```
 if total_mentions == 0:
```

```
 return 0.0
```

Emotion scores: Adoration=+3, Appreciation=+1, Ambivalence=0, Agitation=-1, Anger=-3

```
 eri_raw = sum(
```

```
 float(self.emotion_scores.get(em, 0.0)) * int(ct)
```

```
 for em, ct in emotion_counts.items()
```

```
)
```

```
 eri_mean = eri_raw / total_mentions Range: [-3, +3]
```

Normalize to [-100, +100]

```
 eri_norm = ((float(eri_mean) + 3.0) / 6.0) * 200.0 - 100.0
```

```
eri_norm = float(np.clip(eri_norm, -100.0, 100.0))
```

```
return eri_norm
```

```
```
```

Computing EVI (Emotional Volatility Index):

```
```python
```

```
def _compute_evi(self, n_adore, n_appreciate, n_ambivalent, n_agitate, n_anger):
```

```
 """
```

Computes Emotional Volatility Index (EVI) based on emotion intensity distribution.

Returns EVI in range [0, 100].

```
 """
```

Build list of values according to intensity scale

```
values = (
```

```
 [3] * int(n_adore) +
```

```
 [1] * int(n_appreciate) +
```

```
 [0] * int(n_ambivalent) +
```

```
 [-1] * int(n_agitate) +
```

```
 [-3] * int(n_anger)
```

```
)
```

```
if len(values) == 0:
```

```
 return 0.0
```

```
arr = np.array(values, dtype=float)
```

```
sigma = arr.std(ddof=0) # Population standard deviation
```

```
evi = (sigma / 3.0) * 100.0 # Normalize (max sigma on scale = 3)
```

```
 return float(evi)
```

```
    ````
```

Computing RF (Recency × Frequency):

```
```python
```

Recency: exponential decay based on age in days

```
R = 100.0 * np.exp(-float(age_days) / self.tau_days)
```

Frequency: logarithmic normalization

```
F = min(100.0, (np.log1p(total_mentions) / np.log1p(max_mentions)) * 100.0)
```

Weighted combination

```
RF = (self.rf_weight_r * R) + (self.rf_weight_f * F)
```

```
    ````
```

Classifying Emotional State Band:

```
```python
```

```
def _classify_emotional_state_band(self, eri_tier, evi_tier):
```

```
 """
```

Maps (ERI\_Tier × EVI\_Tier) → Emotional\_State\_Band

Returns: ES1\_Secure\_Loyalty, ES2\_Growth\_Opportunity, ES3\_At\_Risk,

ES4\_Active\_Crisis, ES5\_Critical\_Failure

```
 """
```

ES1: Happy + consistent = advocacy foundation

if eri\_tier in {"Very Positive", "Positive"} and evi\_tier in {"Stable Emotion", "Low Volatility"}:

```
 return "ES1_Secure_Loyalty"
```



ES2: Positive with volatility = engaged but inconsistent

```
if eri_tier in {"Very Positive", "Positive"} and evi_tier in {"Moderate Volatility", "High Volatility", "Extreme Volatility":
```

```
 return "ES2_Growth_Opportunity"
```

ES5: Very Negative always = critical failure

```
if eri_tier == "Very Negative":
```

```
 return "ES5_Critical_Failure"
```

ES4: Negative + any volatility = active crisis

```
if eri_tier == "Negative":
```

```
 return "ES4_Active_Crisis"
```

ES4: Neutral + extreme volatility = chaotic crisis

```
if eri_tier == "Neutral" and evi_tier == "Extreme Volatility":
```

```
 return "ES4_Active_Crisis"
```

ES3: Neutral + other states = at risk (disengagement)

```
if eri_tier == "Neutral":
```

```
 return "ES3_At_Risk"
```

```
return "ES3_At_Risk" Fallback
```

```
...
```

Computing Temporal Intelligence:

```
```python
```

```
def _derive_temporal_intelligence_from_tiers(self, by_tier: dict, all_tiers: list) -> dict:
```

```
    """
```

Assemble structured Temporal Intelligence from tier-level data.

Compares early tiers (Tier_4, Tier_5) vs recent tiers (Tier_1, Tier_2)

Returns dict with named flows:

- ERI_Flow: polarity trajectory
- EVI_Flow: volatility trajectory
- ES_Flow: emotional state band migration
- Emotion_Flow: composition shifts
- Pattern_Flow: interpreted pattern
- Context_Flow: narrative summary
- PEM_Guidance: derived guidance

```
    """
```

```
        recent_tiers = ["Tier_1", "Tier_2"]
```

```
        early_tiers = ["Tier_4", "Tier_5"]
```

```
        Calculate flows
```